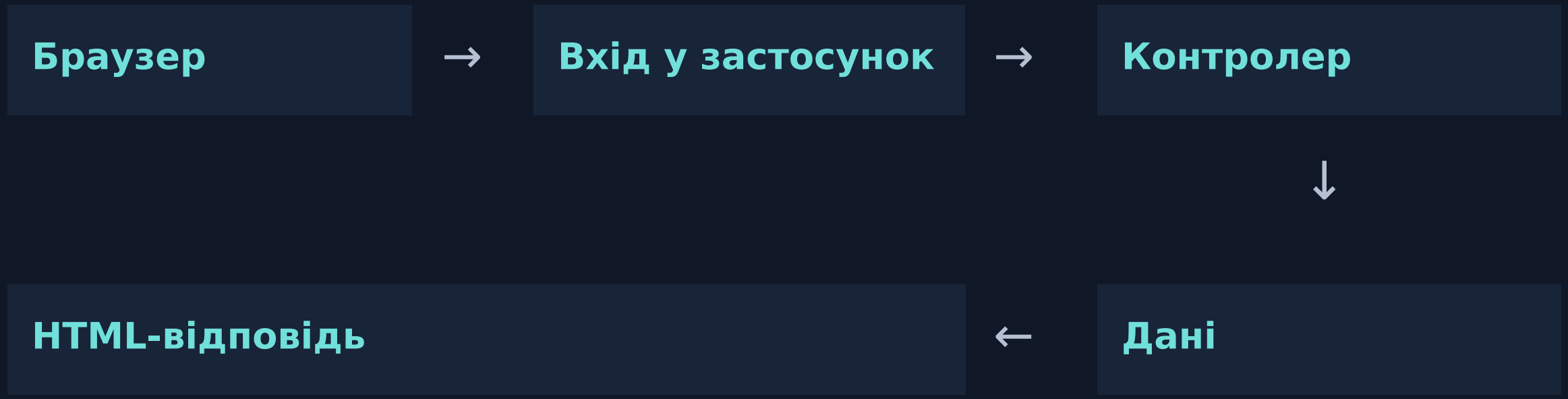
ЛЕКЦІЯ 14

MVC та маршрутизація

Архітектура PHP-застосунку

На прикладі «Менеджера завдань»

Що відбувається після натискання кнопки



Кожен етап має власну відповідальність

Один файл виконує всю роботу

```
$title = trim($_POST['title'] ?? '');
$stmt = $pdo->prepare(
    'INSERT INTO tasks (title) VALUES (?)'
);
$stmt->execute([$title]);
$tasks = $pdo->query('SELECT * FROM tasks');
?>
<h1>Мої завдання</h1>
<?php foreach ($tasks as $task): ?>
  <p><?= htmlspecialchars($task['title']) ?></p>
<?php endforeach; ?>
```

Запит, доступ до даних і HTML опинилися в одному файлі

Ціна змішування відповідальностей

Нова HTML-розмітка	Потрібно орієнтуватися серед SQL і перевірок
Перевірка бізнес-правила	Потрібно відокремлювати його від HTTP і виведення
Нова сторінка	Зростає кількість умов у спільному файлі
Робота в команді	Кілька розробників редагують ті самі ділянки

Відповідальності в MVC

M

Model

Дані
та бізнес-правила

V

View

Відображення даних

C

Controller

Оброблення запиту
та координація

Model: сутність і збереження даних

Task	Назва, статус і правила зміни стану завдання
TaskRepository	Читання та збереження завдань у сховищі
findAll()	Повернути список завдань
add($title, $deadline)	Зберегти нове завдання

View: показ підготовлених даних

```
<h1>Мої завдання</h1>
<?php foreach ($tasks as $task): ?>
  <p>
    <?= htmlspecialchars(
      $task->getTitle(),
      ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8'
    ) ?>
  </p>
<?php endforeach; ?>
```

Шаблон працює з $tasks, які підготував контролер

Controller: координація запиту

```
01  final class TaskController
02  {
03      public function index(): void
04      {
05          $tasks = $this->repository->findAll();
06          $this->render(
07              'tasks/index', ['tasks' => $tasks]
08          );
09      }
10  }
```

Контролер отримує дані й обирає представлення

Структура PHP-проєкту

public/	index.php, assets/	Доступна вебсерверу
src/Controllers/	TaskController.php	Оброблення запитів
src/Models/	Task.php	Сутності й правила
src/Repositories/	TaskRepository.php	Доступ до даних
views/	tasks/, layouts/	HTML-шаблони
vendor/, .env	Залежності та конфігурація	Поза public/

Публічний каталог

Доступ через браузер

`public/index.php`

`public/assets/style.css`

Код поза коренем сайту

`src/`, `views/`, `vendor/`

`.env`

`DocumentRoot = project/public/`

Front Controller

GET / | **POST /tasks** | **GET /tasks/create**

↓

public/index.php

Автозавантаження / Сесія й залежності / Запуск Router

Маршрут складається з методу та шляху

Метод	Шлях	Дія	Призначення
GET	/	index()	Список завдань
GET	/tasks/create	create()	Форма створення
POST	/tasks	store()	Збереження завдання
POST	/tasks/complete	complete()	Зміна статусу

Шлях запиту без параметрів пошуку

```
$method = $_SERVER['REQUEST_METHOD'];
$path = parse_url(
    $_SERVER['REQUEST_URI'], PHP_URL_PATH
) ?: '/';

// /tasks/create?from=menu
// шлях: /tasks/create
// параметри: $_GET['from']
```

Router зіставляє метод і шлях запиту

Таблиця маршрутів у Router

```
01  private array $routes = [];
02
03  public function get(string $path, array $handler): void
04  {
05      $this->routes['GET'][$path] = $handler;
06  }
07
08  public function post(string $path, array $handler): void
09  {
10      $this->routes['POST'][$path] = $handler;
11  }
```

Реєстрація маршруту зберігає обробник, але ще не запускає його

Реєстрація дій TaskController

```
01  $router->get(
02      '/', [TaskController::class, 'index']
03  );
04  $router->get(
05      '/tasks/create', [TaskController::class, 'create']
06  );
07  $router->post(
08      '/tasks', [TaskController::class, 'store']
09  );
10
11  $router->dispatch($method, $path);
```

Оголошення маршрутів передує обробленню поточного запиту

Вибір обробника в dispatch()

```
$handler = $this->routes[$method][$path] ?? null;
if ($handler === null) {
    http_response_code(404);
    echo 'Сторінку не знайдено';
    return;
}
[$controllerClass, $action] = $handler;
$controller = $this->controllers[$controllerClass];
$controller->$action();
```

Навчальний Router використовує точне зіставлення маршрутів

Залежності контролера

```
01  // $pdo уже налаштовано
02  $repository = new TaskRepository($pdo);
03  $controller = new TaskController($repository);
04  $router = new Router([
05      TaskController::class => $controller
06  ]);
07
08  // Конструктор класу Router
09  public function __construct(
10      private array $controllers
11  ) {}
```

Router отримує вже створені контролери з потрібними залежностями

GET /: отримання списку завдань

1	**GET /**	Браузер запитує головну сторінку
2	**Router**	Обирає TaskController::index
3	**Repository**	Повертає список завдань
4	**View**	Формує HTML зі списком

Передача $tasks у шаблон

```
01  $this->render('tasks/index', ['tasks' => $tasks]);
02
03  private function render(string $view, array $data): void
04  {
05      $viewPath = __DIR__ . '/../../views/' . $view . '.php';
06      extract($data, EXTR_SKIP);
07      require $viewPath;
08  }
```

extract() зручний, але шаблон також може читати значення з $data

POST /tasks: оброблення форми

```
01  $title = trim($_POST['title'] ?? '');
02  if ($title === '') {
03      http_response_code(422);
04      $this->render('tasks/create', [
05          'error' => 'Вкажіть назву завдання'
06      ]);
07      return;
08  }
09  $this->repository->add($title, null);
10  header('Location: /', true, 303);
11  exit;
```

Фрагмент store(): перевірка, збереження, перенаправлення

POST, перенаправлення, GET

POST /tasks — Перевірка й збереження

↓

303 Location: / — Відповідь із перенаправленням

↓

GET / — Показ оновленого списку

Сторінка всередині HTML-макета

```
01  // render(): шлях до шаблону вже визначено
02  extract($data, EXTR_SKIP);
03  ob_start();
04  require $viewPath;
05  $content = ob_get_clean();
06  require __DIR__ . '/../../views/layouts/main.php';
07
08  // Усередині views/layouts/main.php
09  <main>
10    <?= $content ?>
11  </main>
```

Шаблон формує фрагмент, а макет вставляє його в <main>

Самоперевірка

01 Де має бути SELECT із бази даних?

02 Хто обирає дію для POST /tasks?

03 Де екранувати назву перед виведенням?

04 Чи обов’язково використовувати extract()?

Пауза для самостійної відповіді

Відповіді та підсумок

Repository	Виконує запит до сховища
Router	Зіставляє метод і шлях з обробником
View	Екранує значення в контексті HTML-виведення
Дані шаблону	Можна читати з $data без extract()

Матеріали до лекції

Лекція 14

surkovkostiantyn.github.io/nmk/programming/lecture_14.html

PHP: extract()

php.net/manual/en/function.extract.php

PHP: htmlspecialchars()

php.net/manual/en/function.htmlspecialchars.php

PHP: буферизація виведення

php.net/manual/en/function.ob-start.php